

ASSIGNMENT-1-

Q1. Difference between Frontend, Backend, and Full-Stack Development

Web development is broadly divided into three areas based on which part of a web application a developer works on.

Frontend Development

Frontend (client-side) development deals with everything the user sees and interacts with directly in the browser — layout, design, colors, buttons, forms, and interactivity. It is built using HTML (structure), CSS (styling), and JavaScript (behavior), often with frameworks such as React, Angular, or Vue.

Real-world example: The visual design, navigation menu, and "Add to Cart" button animations on an e-commerce site like Amazon are built by frontend developers.

Backend Development

Backend (server-side) development handles the logic, database interactions, authentication, and server operations that the user does not see. It is built using languages/frameworks such as Node.js, Python (Django/Flask), Java (Spring), or PHP, along with databases like MySQL or MongoDB.

Real-world example: When a user places an order on Amazon, the backend verifies stock, processes payment, and updates the order database.

Full-Stack Development

A full-stack developer works on both frontend and backend, capable of building a complete, end-to-end web application — from the user interface to the server logic and database.

Real-world example: A single developer building a small startup's website — designing the homepage (frontend) and also writing the login/signup system and database (backend) — is doing full-stack work.

Aspect	Frontend	Backend	Full-Stack
Focus	User interface & experience	Server, logic & database	Both sides combined
Languages	HTML, CSS, JavaScript	Python, Java, Node.js, PHP	Combination of both
Runs on	User's browser	Web server	Both
Example role	UI/UX implementer	API & database engineer	End-to-end builder

Q2. Diagram: Client-Server Model in Web Architecture

The client-server model is the foundation of how the web works. The client (browser) sends a request over the internet to a server, which processes it and sends back a response.

Client-Server Model in Web Architecture

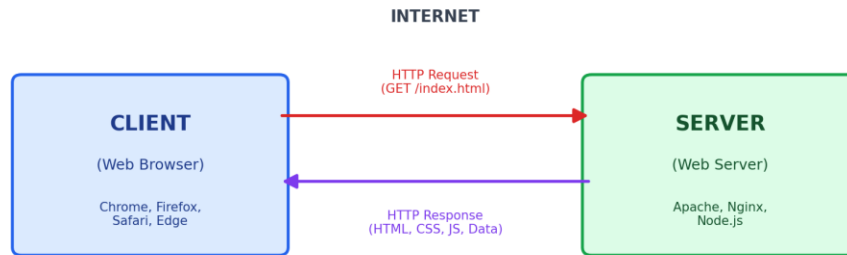


Figure 1: Client-Server communication over HTTP

Step summary: (1) The client sends an HTTP request to the server; (2) the server processes the request (may query a database); (3) the server sends back an HTTP response containing HTML/CSS/JS or data; (4) the browser renders this response for the user.

Q3. How a Browser Requests and Displays a Web Page

When a user types a URL or clicks a link, the browser goes through a well-defined sequence of steps to fetch and render the page:

- DNS Lookup — The browser resolves the domain name (e.g., `www.example.com`) into an IP address using a DNS server.
- Establishing Connection — The browser opens a TCP connection to the server (and performs a TLS handshake if the site uses HTTPS).
- Sending HTTP Request — The browser sends an HTTP GET request asking for the specific page/resource.
- Server Processing — The web server (and possibly a backend/application server and database) processes the request and prepares a response.
- Receiving Response — The server sends back an HTTP response containing the HTML document, along with a status code (e.g., 200 OK).
- Parsing HTML — The browser parses the HTML to build the DOM (Document Object Model).
- Fetching Additional Resources — The browser requests linked resources such as CSS files, JavaScript files, images, and fonts.
- Rendering — The browser combines the DOM with CSS (CSSOM) to build the render tree, calculates layout, and paints pixels on screen.
- JavaScript Execution — Any JavaScript runs, adding interactivity and potentially modifying the DOM dynamically.

This entire process typically takes a few hundred milliseconds to a couple of seconds, depending on network speed and page complexity.

Q4. Tools Required to Set Up a Web Development Environment

A basic but complete web development setup typically includes the following tools:

Tool	Purpose
Code Editor / IDE (e.g., VS Code)	Write, edit, and manage source code with syntax highlighting, extensions, and debugging support.
Web Browser (e.g., Chrome, Firefox)	Run and preview the web page; use built-in Developer Tools to inspect elements, debug JavaScript, and check network requests.
Version Control (Git) & GitHub/GitLab	Track changes to code, collaborate with others, and maintain a history of the project.
Node.js & npm	Run JavaScript outside the browser, manage project dependencies, and use build tools/frameworks (React, Express, etc.).
Package Manager (npm/yarn)	Install and manage third-party libraries and frameworks used in the project.
Web Server (Apache/Nginx/Node Express, or Live Server extension)	Serve web pages locally or in production so they can be accessed via HTTP.
Database (MySQL/MongoDB/PostgreSQL) + client (e.g., MySQL Workbench, MongoDB Compass)	Store, manage, and query application data during backend development.
Browser Developer Tools	Inspect HTML/CSS, debug JavaScript, monitor network calls and performance.
API Testing Tool (Postman/Insomnia)	Test backend APIs by sending requests and checking responses without needing a frontend.
Terminal / Command Line	Run commands for Git, npm, servers, and build scripts.

Q5. What is a Web Server? Examples

A web server is software (often running on dedicated hardware) that stores, processes, and delivers web pages and other resources to clients (browsers) in response to HTTP/HTTPS requests. It listens for incoming requests, retrieves or generates the requested content, and sends it back to the client.

A web server can serve static content (fixed files like HTML, images) directly, or work with an application server to generate dynamic content on the fly.

Commonly Used Web Servers

- Apache HTTP Server — One of the oldest and most widely used open-source web servers, known for flexibility and a large module ecosystem.
- Nginx — A high-performance web server known for handling many concurrent connections efficiently; also used as a reverse proxy and load balancer.

- Microsoft IIS (Internet Information Services) — A web server built for the Windows Server platform, commonly used with ASP.NET applications.
- Node.js (with Express.js) — A JavaScript runtime used to build lightweight, custom web servers, popular for full-stack JavaScript applications.
- LiteSpeed — A high-performance commercial web server often used as a drop-in replacement for Apache, popular in shared hosting environments.

Q6. Roles: Frontend Developer, Backend Developer, Database Administrator

Role	Key Responsibilities
Frontend Developer	Designs and implements the user interface using HTML, CSS, and JavaScript (often with frameworks like React/Angular/Vue); ensures responsiveness, accessibility, cross-browser compatibility, and a smooth user experience; connects the UI to backend APIs to display data.
Backend Developer	Builds server-side logic, APIs, and authentication systems; connects the application to the database; handles business rules, security, and performance of server operations; ensures the application scales and responds correctly to client requests.
Database Administrator (DBA)	Designs, implements, and maintains the database structure (schema); manages data security, backups, and recovery; optimizes queries and indexes for performance; monitors database health, access control, and integrity of stored data.

In a typical project, these roles collaborate closely: the frontend developer builds what the user sees, the backend developer builds the logic that powers it, and the DBA ensures the underlying data is stored safely, efficiently, and reliably.

Q7. Install and Configure VS Code for HTML, CSS & JavaScript

Since this document is a text-based deliverable, an actual screenshot of a local VS Code installation cannot be captured here. Follow these steps on your own machine and insert a screenshot of the final setup into your submission:

- Download VS Code from the official site: <https://code.visualstudio.com/> and install it for your operating system (Windows/macOS/Linux).
- Open VS Code and go to the Extensions view (Ctrl+Shift+X).
- Install "Live Server" — lets you preview HTML pages with auto-reload on save.
- Install "Prettier – Code Formatter" — automatically formats HTML, CSS, and JavaScript code.
- Install "HTML CSS Support" and "JavaScript (ES6) code snippets" for autocompletion and quick snippets.
- (Optional) Install "ESLint" for catching JavaScript errors and enforcing coding standards.
- Create a project folder with an index.html, style.css, and script.js file, then open the folder in VS Code (File > Open Folder).
- Right-click index.html and choose "Open with Live Server" to preview the page in the browser with live reload.
- Take a screenshot showing VS Code with the project folder, an open HTML file, and the Live Server preview running in the browser, then insert that image here in your submitted copy.

[Insert your VS Code setup screenshot here]

Q8. Static vs Dynamic Websites

Aspect	Static Website	Dynamic Website
Content	Fixed; same content shown to every visitor	Changes based on user, input, or data in a database
Technology	HTML, CSS, (minimal JS)	HTML/CSS/JS plus a backend language (PHP, Node.js, Python) and a database
Update process	Requires manually editing and re-uploading files	Content can be updated through a database or admin panel without touching code
Speed	Generally faster to load (no server-side processing)	Can be slightly slower due to server-side processing and database queries
Interactivity	Limited	Highly interactive; supports logins, forms, personalization

Example of a static website: A simple personal portfolio or a company's "About Us" brochure page built with plain HTML and CSS, where the content is the same for every visitor and only changes when the developer edits the files.

Example of a dynamic website: An online shopping site like Flipkart or Amazon, where product listings, prices, and recommendations are pulled from a database and personalized per user, or a social media feed that changes every time it is loaded.

Q9. Five Web Browsers and Their Rendering Engines

Browser	Developer	Rendering Engine
Google Chrome	Google	Blink (forked from WebKit)
Mozilla Firefox	Mozilla Foundation	Gecko
Apple Safari	Apple	WebKit
Microsoft Edge	Microsoft	Blink (Chromium-based, since 2020)
Opera	Opera Software	Blink (Chromium-based)

A rendering engine is the component of a browser responsible for parsing HTML/CSS and painting the page on screen. Although Chrome, Edge, and Opera all use the Blink engine (making them behave very similarly since they share the same open-source Chromium base), Firefox uses Gecko and Safari uses WebKit, which are independently developed engines with their own quirks.

Because of these differences, the same web page can occasionally render slightly differently across browsers — for example, in default font rendering, support for newer CSS features, or handling of certain JavaScript APIs. This is why cross-browser testing is an important part of frontend development, and CSS features are often checked against compatibility tables (such as "Can I use") before relying on them in production.

Q10. Diagram: Basic Web Architecture Flow

The diagram below shows how a typical modern web application flows across four layers — the client, the web server, an API/application server, and a database.

Basic Web Architecture Flow: Client - Server - API - Database

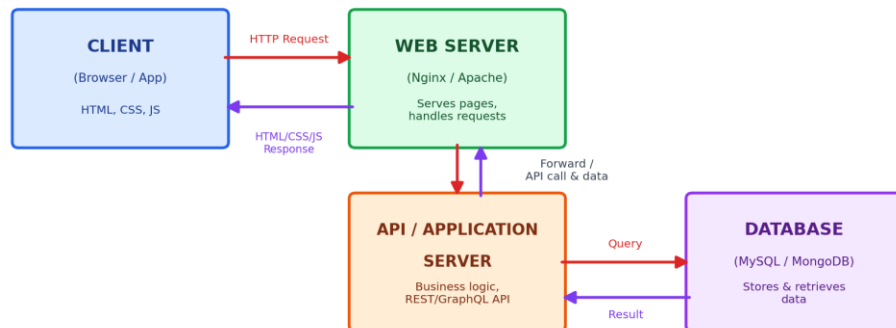


Figure 2: Client → Server → API → Database flow

Flow explanation:

- The Client (browser or mobile app) sends an HTTP request for a page or piece of data.
- The Web Server receives the request; for static content it responds directly, and for dynamic content it forwards the request to the Application/API Server.
- The API/Application Server runs the business logic and, if data is needed, sends a query to the Database.
- The Database stores and retrieves the requested data and returns the result to the API server.
- The API server sends the processed data back through the web server to the client, which renders the final page or updates the UI.